PORTFOLIO · 一个人 · 2026

把事做成，
并让它可被验证。

我不是工程师。我是驱动 AI 干活的人。

四年工业岗位之后离职，现在一个人做。我不亲手画版图，也不自己跑每一条命令——那些反复重复的操作劳动全部交给 AI 和脚本。我负责三件交不出去的事：定目标与边界、判断交出来的东西是真是假、判断什么时候停。

下面这一件事，是我用来证明这三件事的。

1	7 代	6 道	1 条
完整闭环	迭代	质量门	命令
RTL → GDS	每代解决一个问题	每次提交重跑	外行也能证伪

1ouxilisi
github.com/1ouxilisi/sha256-openlane-tapeout

案例

一颗真的做出来了的 SHA-256 芯片

没有商业 EDA 授权，没有流片预算，没有制造排期。OpenLane 1.1.1 + SkyWater sky130A，全开源工具链，从一个 Verilog 文件做到能打开的 GDSII。

项目	结果
最终工作点	55.56 MHz（18 ns），三 PVT corner setup / hold 全正
面积 / 单元	0.4096 mm²（640 × 640 µm die）· 11,475 cells · 利用率 45.77%
物理验证	DRC 0 违例（七代全 0）；Magic 与 KLayout 双 GDS 逐层 XOR，零结构差异
第 7 代做了什么	换掉消息调度结构（16 深循环窗口 → 移位寄存器），同 die 从 41.67 MHz 提到 55.56 MHz（+33%），单元数反而从 12,288 降到 11,475
被它推翻的话	项目早期文档写着「43.9 MHz 是这条 RTL 的上限」——那是旧架构的上限，不是工艺的上限

没做成的部分也在这页上。

v1–v4 时序没收敛（v4 nom 角 setup 余量 -7.63 ns，按签核标准不合格）；裸综合对照与 OpenLane 结论符号相反；功耗角名一度标错（6.84 mW 是 Typical 角，不是最保守的 Fastest）。这些不是丢人的地方，是判断力的来源。

凭什么可信

不是「我写它通过它就通过」

三条互不依赖的证据链。任一条断了，结论就站不住。

① 结构可信

仓库每次 push，在干净的 Ubuntu runner 上重跑 6 道门：v6 / v7 功能回归、独立向量校验、两代静态 lint、驱动级三方对照。任何人都能看到跑没跑过、绿没绿。

② 数字可信

期望摘要由 Python hashlib 独立计算，不用我自己写的常量；两个版图工具出的 GDS 逐层 XOR，不一致直接报错。「自己验自己」被结构性地禁掉。

③ 外人可证伪

不需要懂芯片，也不需要白皮书。三条命令，跑不通就是假的——

```
bash scripts/verify.sh            # ■■■■
bash scripts/verify_driver.sh      # C ■■ -> ■ RTL -> hashlib
python3 scripts/check_vectors.py   # ■■■■■■■■
```

最后一层是把 C 主机驱动打进验证链：C 驱动打出寄存器写序列 → Verilog 主机模型原样发到真 RTL 的 AXI4-Lite 总线上 → hashlib 算期望值。三份实现互不依赖，四条消息全过（含 119 字节两块流式）。这一层同时回答了「别人能不能接手」。

03 / BOUNDARY

边界

我负责什么，不负责什么

边界写清楚，比能力写得漂亮更有用。

我负责	我不负责
定目标、划边界、切里程碑	具体版图细节、每一行 RTL
判断交付物真假	把自己变成工程师
把重复劳动交给机器	亲手跑每条命令、盯每个日志
判断什么时候停	无限逼近「更完美」

可迁移

能搬走的不是知识，是验收结构

领域	证据载体
芯片	GDS + 多角 STA 报告 + DRC + 双工具 XOR
机器人动作包	关节轨迹 + 正运动学复算 + 真机/仿真同帧对比
短剧 / 内容	成片 + 分镜与成片逐镜一致 + 素材授权链
汽修手册	车型 / 年款 / 扭矩值可溯源 + 实车一步一验
财务分析	每格数字回原始报表 + 口径写明 + 可复算

怎么干活

一个人的五步，重复劳动全部外包

既然身份是「驱动 AI 干活的人」，该展示的就不是我会多少知识，而是我每次怎么组织一次工作。

1 定目标

一句话说清要什么。「做出一个能打开的 GDS」，不是「研究一下」——目标得能被别人复述。

2 划边界

同时说清不做什么：不量产、不做 DFT 扫链、不追完美。边界不写，工期就没有终点。

3 全权交出去

跑流程、读日志、写脚本、渲染、排版——重复劳动全交给机器。人只盯两件事：还剩什么、结论对不对。

4 判真假

只认互不依赖的证据，不认「我跑通了」。三条：独立复算、双工具互验、外人能一条命令证伪。

5 决定停还是继续

每代只改一个变量，改完先判断收益，再决定要不要下一代。不靠惯性往前推。

三个真实事例，都在同一件事里

当时以为	实际是
功耗 6.84 mW 取的是最保守的 Fastest 角	其实是 Typical 角。数字没错，标签错了——比最坏情况乐观约 14%。
这条 RTL 到 43.9 MHz 就到底了	那是旧架构的上限。第 7 代换掉调度结构后做到 55.56 MHz。
四条消息跑通就算接口没问题	还得在陌生机器上用一条命令复现。CI 第 6 道门干的就是这件事。

我不做的事也写清楚：不亲自写每行 RTL，不盯每个日志，不为了一个更漂亮的数字再加一代。

04 / CONTACT

找我做什么

只做能交付、且能被验证的事

适合找我

- 一件模糊的事，需要有人定目标、切里程碑，然后把它跑成可验证的交付。
- 你的团队已经在用 AI 干活，但没人判断交付真假——我帮你搭验收结构。
- 需要一个「从零到能打开、能检查」的完整案例，而不是一份方案。

不适合找我

- 需要资质签字、需要承担法律责任的鉴定与认证。
- 需要长期驻场，或需要我假装是科班工程师。
- 只需要「看起来很像」的素材，不需要真的能跑。

1ouxilisi

一个人，不是团队。署名之外的任何头衔都不必加。

作品集	`https://1ouxilisi.github.io/sha256-openlane-tapeout/`
项目详情	`.../project.html`
验收手册	`.../VERIFYING.md`
代码	`github.com/1ouxilisi/sha256-openlane-tapeout`
镜像	`gitee.com/beyond-invincibility/sha256-openlane-tapeout`

本页所有数字都能在仓库对应日志里查到。

跑不通的命令，欢迎直接说它是假的。